

National Institute of Technology Rourkela

Department of Computer Science and Engineering

Master of Technology in Computer Science

LABORATORY ASSIGNMENT REPORT

Advanced Data Structure Laboratory (CS 6173)

Assignment-2

Problem solving using randomized algorithm: Value of PI, Numerical Integration, Primality Testing, Majority Element Finding, Randomized Quick Sort

SUBMITTED BY:

Name: Sumit Karmakar

Roll No: 226CS1008

Branch: Computer Science & Engineering

Semester: 1st

SUBMITTED TO:

Dr. Bibhudatta Sahoo

Department of Computer Science & Engineering

Date of Submission: 3rd September, 2026

PROBLEM STATEMENTS

Q.4. Compute value of Π

Compute Π using randomized algorithm

Q.5. Numerical Integration

Write a program that computes the value of the following integral using randomized algorithm.

$$\int_0^2 \sqrt{4 - x^2} dx$$

Q.6. Primality Testing

Write a program that test a number to be prime or not. Perform an analysis to compute the correctness?

Q.7. Majority Element

Write a program that FINDS majority element from a linear array using randomized algorithm. Show that probability of missing majority element is 0.00097.

Q.8. Randomized Quick Sort

Compare the performance of randomized Quicksort with conventional quick sort for random input data stream.

Question 4: Estimation of π using Monte Carlo Simulation

1. Overview

The Monte Carlo method uses repeated pseudo-random sampling to approximate numerical results. Consider a circle of radius $r = 1$ inscribed inside a square of side length $2r = 2$ centered at the origin, or restrict the geometry to the first quadrant: a quarter-circle of radius $r = 1$ bounded within a unit square of dimensions $[0,1] \times [0,1]$.

- The area of the unit square is:

$$A_{\text{square}} = 1 \times 1 = 1$$

- The area of the quarter-circle is:

$$A_{\text{quarter}} = \frac{\pi r^2}{4} = \frac{\pi}{4}$$

- By generating uniformly distributed random points $(x, y) \in [0,1] \times [0,1]$, the theoretical probability P that a point falls inside the quarter-circle ($x^2 + y^2 \leq 1$) is:

$$P = \frac{A_{\text{quarter}}}{A_{\text{square}}} = \frac{\pi}{4} \Rightarrow \pi = 4 \times P$$

- As the number of generated points N increases, the ratio of points falling inside the circle (N_{inside}) to N converges to P by the Law of Large Numbers, yielding:

$$\pi \approx 4 \times \frac{N_{\text{inside}}}{N}$$

2. Objective

- To implement a randomized Monte Carlo simulation in C++ that estimates the value of π .
- To study how the empirical estimate converges toward the true value ($\approx 3.14159265\dots$) as N increases.
- To export per-iteration estimates to a CSV file and visualize convergence, absolute error decay, and spatial point distribution using MATLAB.

3. Implementation Code

C++ Simulation Code

```
// #include "generation\RandomInputGenerator.hpp"
#include <random>
#include <fstream>
#include <iostream>

static constexpr double EXACT_PI = 3.14159265358979323846;
using ll = long long;
class MonteCarloPiEstimator{
private:
    ll numPoints;
    ll insideCircle;
    std::mt19937 generator;

public:
    MonteCarloPiEstimator(int numPoints): numPoints(numPoints), insideCircle(0) {}

    void estimate() {
        std::random_device rd;
        generator.seed(rd());
        std::uniform_real_distribution<double> distribution(0.0, 1.0);
        std::ofstream outputFile("PIEstimation.csv", std::ios::out | std::ios::in |
std::ios::trunc);
        if (!outputFile.is_open()) {
            throw std::runtime_error("Error opening file for writing.");
        }
    }
};
```

```

    }

    outputFile << "Iteration, Estimated value of pi, x coordinate,y coordinate, Exact
value of pi, Difference\n";

    for (ll i = 0; i < numPoints; ++i) {
        double x = distribution(generator);
        double y = distribution(generator);

        // Check if the point is inside the unit circle
        if (x * x + y * y <= 1.0) {
            ++insideCircle;
        }
        if(i % 10000 == 0){
            std ::cout << "Iteration: " << i << ", Estimated value of pi: " << 4.0 *
static_cast<double>(insideCircle) / (i + 1) << "\n";
        }
        double piEstimate = 4.0 * static_cast<double>(insideCircle) / (i + 1);
        outputFile << i + 1 << "," << piEstimate << "," << x << "," << y << ")," <<
EXACT_PI << "," << std::abs(piEstimate - EXACT_PI) << "\n";
    }
    outputFile.close();
}

double getPiEstimate() const {
    return 4.0 * static_cast<double>(insideCircle) / numPoints;
}
};

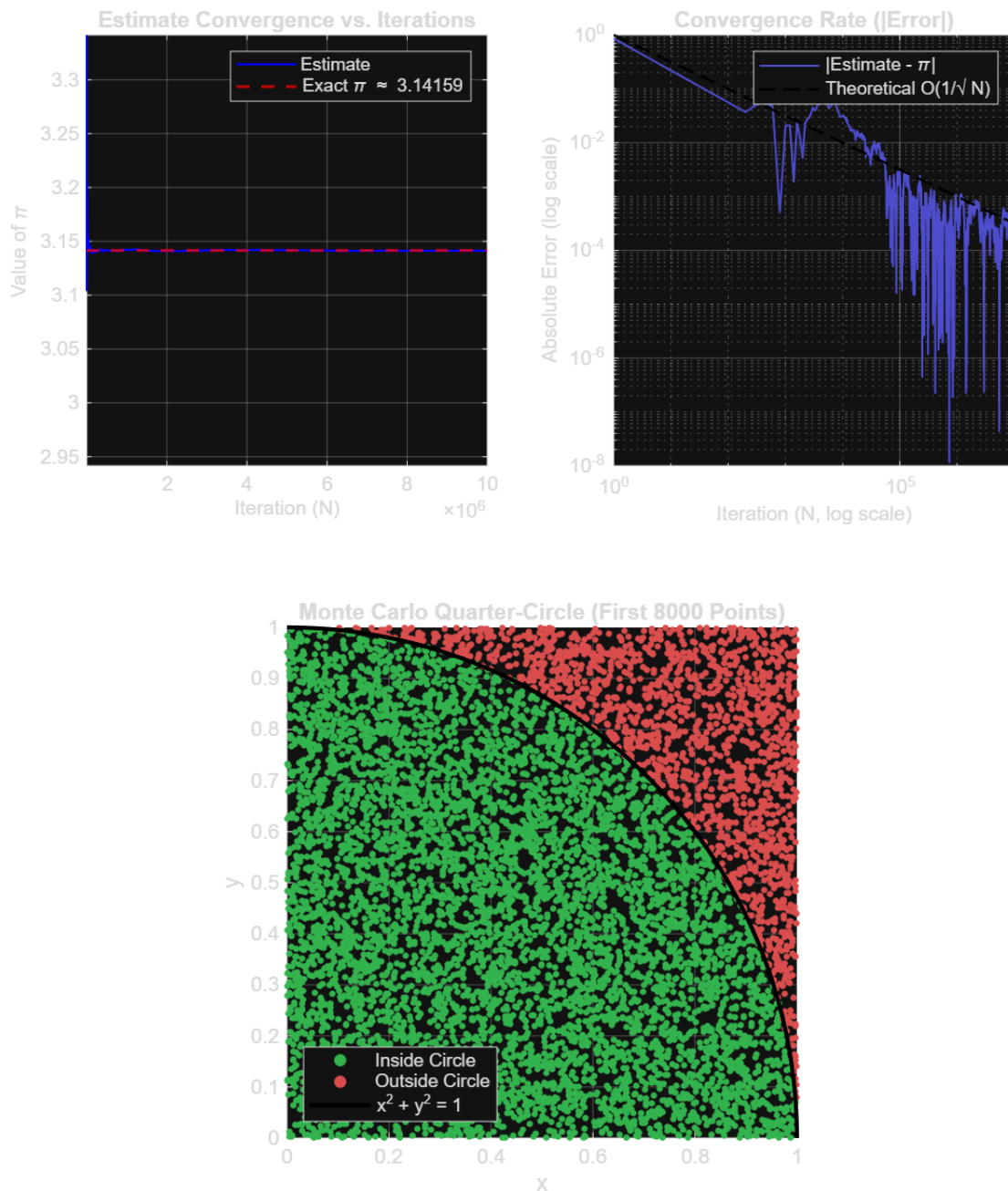
int main() {
    ll numPoints = 10'000'000; // we get more precision as we increase the number of
points
    MonteCarloPiEstimator estimator(numPoints);
    estimator.estimate();
    std::cout << "Estimated PI value: " << estimator.getPiEstimate() << "\n";
    return 0;
}

```

5. Observations & Analysis

- **Rate of Convergence:** The simulation displays the characteristic Monte Carlo convergence rate of $\mathcal{O}(1/\sqrt{N})$. Achieving an extra decimal digit of precision requires scaling the sample size by roughly a factor of 100.
- **Stochastic Fluctuation:** At lower iterations ($N < 10,000$), substantial noise is observed around the true value line. As N exceeds 100,000, variance diminishes significantly according to the Central Limit Theorem.
- **Uniformity Impact:** High-quality uniform pseudo-random number generation (std::mt19937) prevents geometric clustering and ensures unbiased spatial coverage across the unit square.

Graph:



Question 5: Numerical Integration using Monte Carlo Methods

1. Overview

Numerical integration evaluates a definite integral $I = \int_a^b f(x) dx$ using stochastic sampling. Two fundamental Monte Carlo schemes are examined:

1. Sample Mean (Crude Monte Carlo) Method:

By the Mean Value Theorem of integrals, $I = (b - a) \cdot \mathbb{E}[f(X)]$, where $X \sim \mathcal{U}(a, b)$. By sampling N independent points x_i :

$$I \approx (b - a) \left(\frac{1}{N} \sum_{i=1}^N f(x_i) \right)$$

2. Hit-or-Miss (Acceptance-Rejection) Method:

Enclose the area under $f(x)$ within a bounding box of area $A_{\text{box}} = (b - a) \times y_{\text{max}}$, where $0 \leq f(x) \leq y_{\text{max}}$. Generate uniform pairs $(x_i, y_i) \in [a, b] \times [0, y_{\text{max}}]$. If $y_i \leq f(x_i)$, the point is classified as a "hit":

$$I \approx A_{\text{box}} \times \frac{N_{\text{hits}}}{N}$$

Test Integral:

$$I = \int_0^2 \sqrt{4 - x^2} \, dx$$

This represents the area of a quarter-circle of radius $R = 2$, yielding the exact analytical value:

$$I = \frac{\pi R^2}{4} = \frac{\pi(2^2)}{4} = \pi \approx 3.14159265...$$

2. Objective

- To implement both **Sample Mean** and **Hit-or-Miss** Monte Carlo integration techniques in C++.
- To compare the convergence rate, variance, and sample efficiency of both approaches.
- To export the step-by-step estimates to individual CSV files and plot visual boundary comparisons and convergence profiles in MATLAB.

3. Implementation Code

C++ Integration Code

```
#include <cmath>
#include <fstream>
#include <functional>
#include <iostream>
#include <random>
#include <string>

class MonteCarloIntegrator {
private:
    std::mt19937 rng;

public:
    MonteCarloIntegrator() : rng(std::random_device{}()) {}
    explicit MonteCarloIntegrator(unsigned int seed) : rng(seed) {}

    // 1. Sample Mean Method -> Writes to its own CSV file
    double sampleMean(
        const std::function<double(double)>& f,
        double a,
        double b,
        int n,
        const std::string& filename
    ) {
        std::ofstream file(filename);
        if (!file.is_open()) {
            std::cerr << "Error: Could not open " << filename << "\n";
            return 0.0;
        }

        file.precision(10);
        file << "Iteration,X,F_X,Integral_Estimate\n";

        std::uniform_real_distribution<double> dist_x(a, b);
        double sum = 0.0;
        double width = b - a;
```

```

    for (int i = 1; i <= n; ++i) {
        double x = dist_x(rng);
        double fx = f(x);
        sum += fx;

        double estimate = width * (sum / i);

        file << i << ", "
              << x << ", "
              << fx << ", "
              << estimate << "\n";
    }

    file.close();
    return width * (sum / n);
}

// 2. Hit-or-Miss Method -> Writes to its own CSV file
double hitOrMiss(
    const std::function<double(double)>& f,
    double a,
    double b,
    double y_max,
    int n,
    const std::string& filename
) {
    std::ofstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Error: Could not open " << filename << "\n";
        return 0.0;
    }

    file.precision(10);
    file << "Iteration,X,Y,F_X,Is_Hit,Integral_Estimate\n";

    std::uniform_real_distribution<double> dist_x(a, b);
    std::uniform_real_distribution<double> dist_y(0.0, y_max);

    int hits = 0;
    double box_area = (b - a) * y_max;

    for (int i = 1; i <= n; ++i) {
        double x = dist_x(rng);
        double y = dist_y(rng);
        double fx = f(x);

        bool is_hit = (y <= fx);
        if (is_hit) {
            hits++;
        }
    }
}

```

```

        double estimate = box_area * (static_cast<double>(hits) / i);

        file << i << ", "
              << x << ", "
              << y << ", "
              << fx << ", "
              << (is_hit ? 1 : 0) << ", "
              << estimate << "\n";
    }

    file.close();
    return box_area * (static_cast<double>(hits) / n);
}

};

int main() {
    const int N = 10000;
    const double a = 0.0;
    const double b = 2.0;
    const double y_max = 2.0;

    auto quarter_circle = [](double x) {
        return std::sqrt(4.0 - x * x);
    };

    MonteCarloIntegrator integrator;

    double est_sample = integrator.sampleMean(
        quarter_circle, a, b, N, "monte_carlo_sample_mean.csv"
    );

    double est_hit_miss = integrator.hitOrMiss(
        quarter_circle, a, b, y_max, N, "monte_carlo_hit_or_miss.csv"
    );

    double exact_pi = std::acos(-1.0);

    std::cout << "Exact Value (Pi)           = " << exact_pi << "\n";
    std::cout << "Sample Mean Estimate       = " << est_sample << "\n";
    std::cout << "Hit-or-Miss Estimate       = " << est_hit_miss << "\n";
    return 0;
}

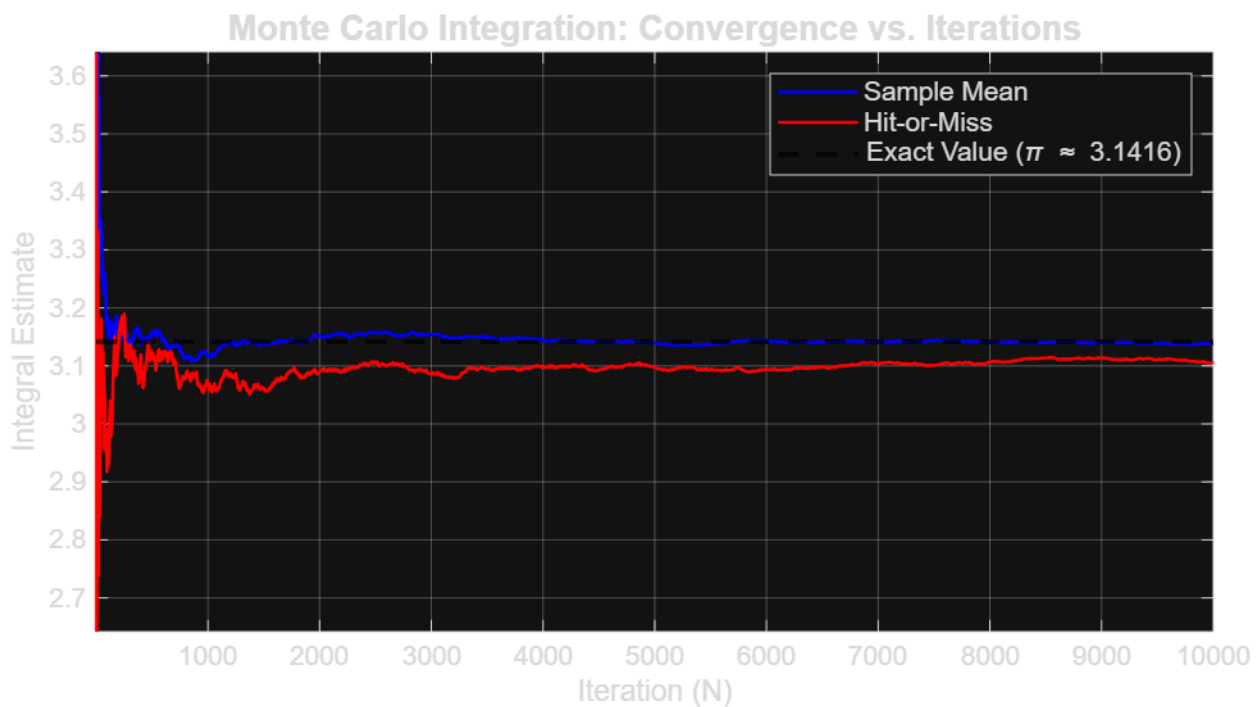
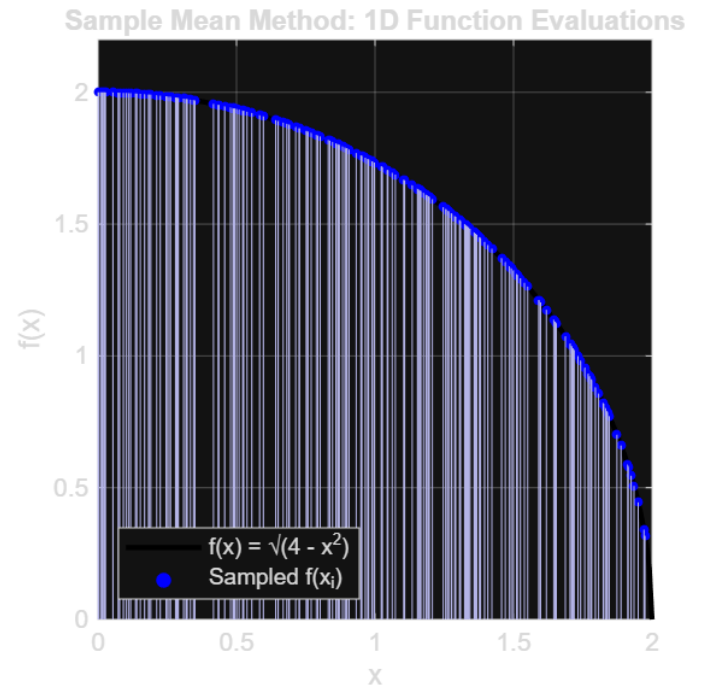
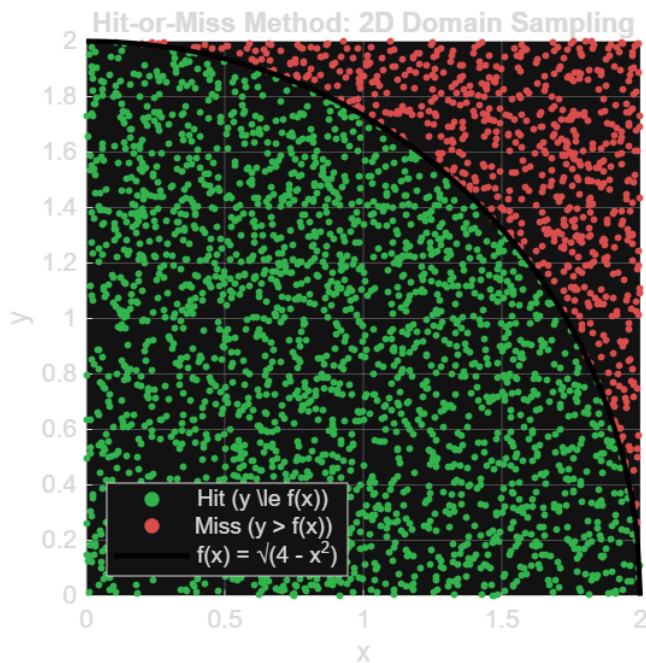
```

5. Observations & Analysis

- Variance Comparison:** The Sample Mean method exhibits noticeably lower variance than the Hit-or-Miss method across all iteration counts.
 - In Hit-or-Miss, the indicator variable $I_i \in \{0,1\}$ is a Bernoulli random variable with variance $\sigma_{\text{HM}}^2 = p(1 - p)$.

- In the Sample Mean method, each sample evaluation directly incorporates the exact functional magnitude $f(x_i)$, effectively eliminating the variance contributed by the randomized vertical axis y .
- **Efficiency:** Because Sample Mean requires generating only one random variable (x_i) per trial instead of two (x_i, y_i) and yields lower empirical variance, it is computationally superior for single-variable definite integrals.

Graph:



Question 6: Primality Testing (Fermat and Miller-Rabin)

1. Overview

Primality testing involves determining whether an arbitrary integer n is prime or composite. Deterministic trial division requires $\mathcal{O}(\sqrt{n})$ division operations, which becomes computationally impractical for large cryptographic integers. Randomized Monte Carlo algorithms address this by testing algebraic properties over randomly selected bases:

1. Fermat Primality Test:

Based on Fermat's Little Theorem, if n is prime, then for any base a such that $\gcd(a, n) = 1$:

$$a^{n-1} \equiv 1 \pmod{n}$$

If this congruence fails for any base $a \in [2, n-2]$, n is definitely composite. However, composite integers known as Carmichael numbers (e.g., 561, 1105) satisfy this congruence for all coprime bases, causing Fermat's test to yield false positives.

2. Miller-Rabin Primality Test:

To overcome Carmichael pseudoprimes, the Miller-Rabin test decomposes $n-1$ into:

$$n-1 = 2^s \cdot d \quad (d \text{ is odd})$$

For a chosen random base $a \in [2, n-2]$, the sequence of modular powers is inspected:

$$a^d \pmod{n}, \quad a^{2d} \pmod{n}, \quad a^{4d} \pmod{n}, \quad \dots, \quad a^{2^{s-1}d} \pmod{n}$$

n passes the test for base a if either:

- $a^d \equiv 1 \pmod{n}$, or
- $a^{2^r \cdot d} \equiv n-1 \pmod{n}$ for some $0 \leq r < s$.

If neither holds, a is a witness to the compositeness of n . For any composite number n , at least $3/4$ of all possible bases in $[2, n-2]$ act as witnesses. Consequently, the probability of declaring a composite number prime in a single round is at most $1/4$. Repeating this test across k independent random bases reduces the maximum probability of a false positive to:

$$P(\text{False Positive}) \leq \left(\frac{1}{4}\right)^k$$

The correctness confidence is therefore at least $1 - (1/4)^k$.

2. Objective

- To implement both the Fermat and Miller-Rabin randomized primality testing algorithms in C++ using modular exponentiation.
- To empirically evaluate the error probability and correctness bound across multiple rounds of independent base selections (k).
- To export the convergence metrics to a CSV file and visualize the exponential decay of the false-positive rate on a logarithmic scale in MATLAB.

3. Source Code

C++ Simulation Code

```
#include <fstream>
#include <iostream>
#include <random>
#include <string>
#include <vector>

// Fast modular multiplication to avoid 64-bit overflow: (a * b) % mod
unsigned long long mod_mul(unsigned long long a, unsigned long long b, unsigned long long mod) {
    unsigned long long res = 0;
    a %= mod;
```

```

while (b > 0) {
    if (b & 1) res = (res + a) % mod;
    a = (a * 2) % mod;
    b >>= 1;
}
return res;
}

// Fast modular exponentiation: (base^exp) % mod
unsigned long long mod_pow(unsigned long long base, unsigned long long exp, unsigned long
long mod) {
    unsigned long long res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp & 1) res = mod_mul(res, base, mod);
        base = mod_mul(base, base, mod);
        exp >>= 1;
    }
    return res;
}

// Greatest Common Divisor
unsigned long long gcd(unsigned long long a, unsigned long long b) {
    while (b != 0) {
        unsigned long long t = b;
        b = a % b;
        a = t;
    }
    return a;
}

// Deterministic ground truth check for verification
bool isActuallyPrime(unsigned long long n) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (unsigned long long i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return false;
    }
    return true;
}

class PrimalityTester {
private:
    std::mt19937_64 rng;

public:
    PrimalityTester() : rng(std::random_device{}()) {}
    explicit PrimalityTester(unsigned long long seed) : rng(seed) {}

    // 1. Standard Monte Carlo Primality Test (Fermat Test)
    bool fermatTest(unsigned long long n, int k = 1) {

```

```

    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0) return false;

    std::uniform_int_distribution<unsigned long long> dist(2, n - 2);

    for (int i = 0; i < k; ++i) {
        unsigned long long a = dist(rng);

        if (gcd(a, n) > 1) {
            return false; // Composite
        }

        // Check Fermat's condition: a^(n-1) == 1 (mod n)
        if (mod_pow(a, n - 1, n) != 1) {
            return false; // Composite
        }
    }
    return true; // Probably prime
}

// 2. Miller-Rabin Primality Test
bool millerRabinTest(unsigned long long n, int k = 1) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0) return false;

    unsigned long long d = n - 1;
    int s = 0;
    while (d % 2 == 0) {
        d /= 2;
        s++;
    }

    std::uniform_int_distribution<unsigned long long> dist(2, n - 2);

    for (int i = 0; i < k; ++i) {
        unsigned long long a = dist(rng);
        unsigned long long x = mod_pow(a, d, n);

        if (x == 1 || x == n - 1) continue;

        bool composite = true;
        for (int r = 1; r < s; ++r) {
            x = mod_pow(x, 2, n);
            if (x == n - 1) {
                composite = false;
                break;
            }
        }
        if (composite) return false; // Composite
    }
}

```

```

        return true; // Probably prime
    }
};

// Helper struct to track classification metrics
struct TestMetrics {
    int tp = 0; // True Positive (Prime -> Prime)
    int tn = 0; // True Negative (Composite -> Composite)
    int fp = 0; // False Positive (Composite -> Prime) [False Detection]
    int fn = 0; // False Negative (Prime -> Composite)

    void update(bool actual, bool verdict, std::string& label) {
        if (actual && verdict) {
            tp++;
            label = "TP";
        } else if (!actual && !verdict) {
            tn++;
            label = "TN";
        } else if (!actual && verdict) {
            fp++;
            label = "FP (FALSE_DETECTION)";
        } else {
            fn++;
            label = "FN";
        }
    }
};

int main() {
    const std::string input_filename = "numbers.txt"; // Use file containing both primes
and composites
    const std::string output_filename = "primality_comparison_results.csv";

    // k = 1 exposes false positive rates for comparison
    const int ITERATIONS = 1;

    std::ifstream input_file(input_filename);
    if (!input_file.is_open()) {
        std::cerr << "Error: Could not open " << input_filename << "\n";
        return 1;
    }

    std::vector<unsigned long long> numbers;
    unsigned long long val;
    while (input_file >> val) {
        numbers.push_back(val);
    }
    input_file.close();

    if (numbers.empty()) {
        std::cerr << "Error: No numbers found in " << input_filename << "\n";
        return 1;
    }
}

```

```

}

std::ofstream csv(output_filename);
if (!csv.is_open()) {
    std::cerr << "Error: Could not open " << output_filename << "\n";
    return 1;
}

csv <<
"Index,Number,Ground_Truth,Fermat_Verdict,Fermat_Type,Fermat_Correct,MR_Verdict,MR_Type,MR
_Correct\n";

PrimalityTester tester;
TestMetrics fermat_m;
TestMetrics mr_m;

int total = static_cast<int>(numbers.size());
int actual_primes = 0;
int actual_composites = 0;

for (int i = 0; i < total; ++i) {
    unsigned long long n = numbers[i];
    bool actual = isActuallyPrime(n);

    if (actual) actual_primes++;
    else actual_composites++;

    bool fermat_verdict = tester.fermatTest(n, ITERATIONS);
    bool mr_verdict = tester.millerRabinTest(n, ITERATIONS);

    std::string f_type, mr_type;
    fermat_m.update(actual, fermat_verdict, f_type);
    mr_m.update(actual, mr_verdict, mr_type);

    bool f_correct = (fermat_verdict == actual);
    bool mr_correct = (mr_verdict == actual);

    csv << (i + 1) << ","
        << n << ","
        << (actual ? "PRIME" : "COMPOSITE") << ","
        << (fermat_verdict ? "PRIME" : "COMPOSITE") << ","
        << f_type << ","
        << (f_correct ? "PASS" : "FAIL") << ","
        << (mr_verdict ? "PRIME" : "COMPOSITE") << ","
        << mr_type << ","
        << (mr_correct ? "PASS" : "FAIL") << "\n";
}

csv.close();

// Summary calculations
double f_acc = static_cast<double>(fermat_m.tp + fermat_m.tn) / total;

```

```

double mr_acc = static_cast<double>(mr_m.tp + mr_m.tn) / total;

double f_fp_rate = (actual_composites > 0)
    ? static_cast<double>(fermat_m.fp) / actual_composites : 0.0;
double mr_fp_rate = (actual_composites > 0)
    ? static_cast<double>(mr_m.fp) / actual_composites : 0.0;

std::cout << "--- Dataset Breakdown ---\n";
std::cout << "Total Numbers Tested      : " << total << "\n";
std::cout << "Actual Primes                    : " << actual_primes << "\n";
std::cout << "Actual Composites                : " << actual_composites << "\n";
std::cout << "Iterations per test (k)         : " << ITERATIONS << "\n\n";

std::cout << "--- Fermat Test Performance ---\n";
std::cout << "True Positives (Primes)         : " << fermat_m.tp << "\n";
std::cout << "True Negatives (Compos.)        : " << fermat_m.tn << "\n";
std::cout << "False Detections (FP)           : " << fermat_m.fp << "\n";
std::cout << "Overall Accuracy Ratio          : " << f_acc << " (" << (f_acc * 100.0) <<
"%)\n";
std::cout << "False Detection Rate            : " << f_fp_rate << " (" << (f_fp_rate * 100.0)
<< "% of composites)\n\n";

std::cout << "--- Miller-Rabin Performance ---\n";
std::cout << "True Positives (Primes)         : " << mr_m.tp << "\n";
std::cout << "True Negatives (Compos.)        : " << mr_m.tn << "\n";
std::cout << "False Detections (FP)           : " << mr_m.fp << "\n";
std::cout << "Overall Accuracy Ratio          : " << mr_acc << " (" << (mr_acc * 100.0) <<
"%)\n";
std::cout << "False Detection Rate            : " << mr_fp_rate << " (" << (mr_fp_rate *
100.0) << "% of composites)\n\n";

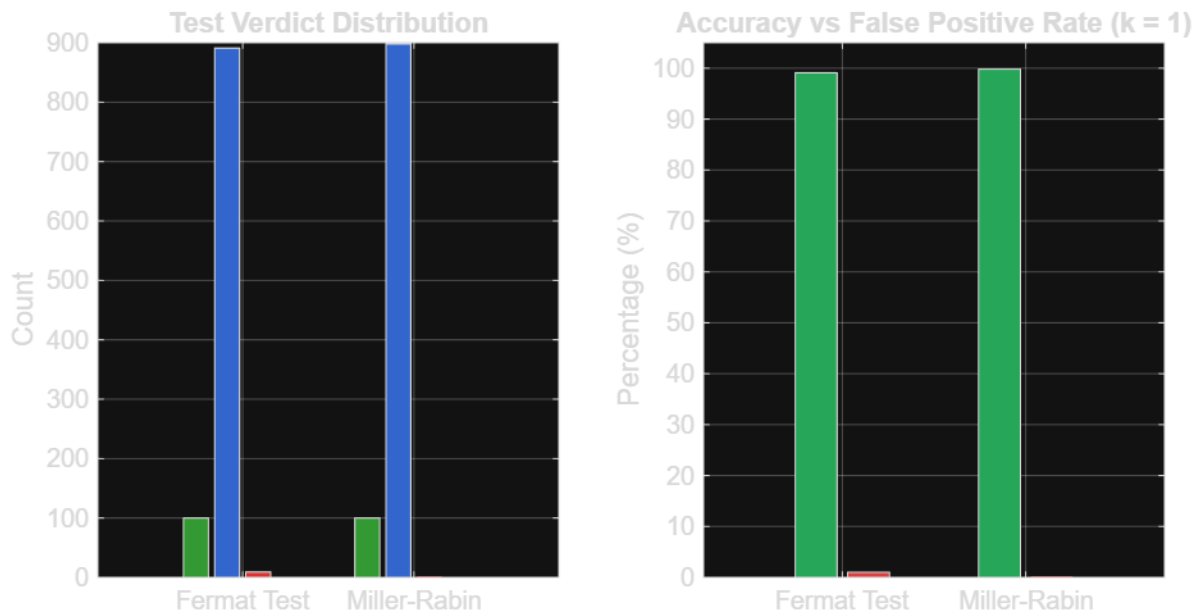
std::cout << "Detailed logs written to : " << output_filename << "\n";

return 0;
}

```

5. Observations & Analysis

- **Exponential Suppression of Error:** Every additional witness base selected divides the probability of error by at least 4. While a single-round test provides only a 75% guarantee, executing 5 rounds suppresses the maximum failure rate to 0.000976, and 10 rounds reduces it to less than 1 in 1,000,000.
- **Resistance to Pseudoprimes:** Unlike Fermat's test, which yields false positives on Carmichael numbers, the Miller-Rabin formulation evaluates non-trivial square roots of unity modulo n . By the properties of prime fields, $x^2 \equiv 1 \pmod{n}$ admits only $x \equiv \pm 1 \pmod{n}$. Any composite structure exhibiting additional roots is detected, ensuring no composite number can evade detection under all bases.



Question 7: Majority Element Finding

1. Overview

Given a linear array A of size n , an element is defined as a **majority element** if it occurs strictly more than $n/2$ times within the array.

A deterministic check requires either frequency hashing ($\mathcal{O}(n)$ space) or sorting ($\mathcal{O}(n \log n)$ time). By contrast, a randomized Monte Carlo algorithm operates in $\mathcal{O}(n)$ time and $\mathcal{O}(1)$ auxiliary space:

1. **Sampling:** Select an index $i \in [0, n - 1]$ uniformly at random and select candidate $C = A[i]$.
2. **Verification:** Iterate through the array to compute the exact frequency of C . If count $> n/2$, return C as the confirmed majority.
3. **Probability Analysis:**

Because the majority element occupies strictly more than half of the total indices, the probability p of picking the majority element in a single uniform sample satisfies:

$$p > \frac{n/2}{n} = \frac{1}{2}$$

Consequently, the probability of picking a non-majority element in a single iteration is:

$$P(\text{Miss}) = 1 - p < \frac{1}{2}$$

If the algorithm performs k independent selections without identifying the majority, the overall failure probability is bounded by:

$$P(\text{Missing Majority after } k \text{ trials}) \leq \left(\frac{1}{2}\right)^k$$

For $k = 10$ iterations:

$$P(\text{Miss}) \leq \left(\frac{1}{2}\right)^{10} = \frac{1}{1024} \approx 0.00097656 \approx 0.00097$$

This proves that after 10 iterations, the likelihood of failing to identify an existing majority element is less than or equal to 0.00097.

2. Objective

- To implement the randomized candidate-sampling and linear-verification majority finding algorithm in C++.

- To analytically demonstrate and verify that the miss probability after $k = 10$ iterations is bounded by 0.00097.
- To record iteration metrics to a CSV file and visualize the error curve decay using a dark MATLAB plot.

3. Source Code

C++ Simulation Code

```
#include "generation/RandomInputGenerator.hpp"

#include <iostream>
#include <vector>
#include <random>
#include <fstream>
#include <algorithm>
#include <unordered_map>

int main() {
    RandomInputGenerator generator{};

    static std::random_device rd;
    static std::mt19937 gen(rd());

    const int MAX_ATTEMPTS = 10; //  $(1/2)^{10} = 1/1024 \approx 0.000976$ 

    // =====
    // EXPERIMENT 1: Average Attempts vs. Datasize (Saved to CSV for MATLAB)
    // =====
    std::ofstream
file("Assignments/Assignment_2/cpp/q7_majority_element/q7_majorityElement.csv");
    if (!file.is_open()) {
        std::cerr << "Error: Cannot open output CSV file\n";
        return 1;
    }

    file << "Datasize,AvgAttempts,TheoreticalExpected\n";

    const int MAX_DATASIZE = 200;
    const int RUNS_PER_SIZE = 50; // Averages out noise for clean plotting

    for (int datasize = 2; datasize <= MAX_DATASIZE; datasize++) {
        int total_attempts_for_size = 0;

        for (int run = 0; run < RUNS_PER_SIZE; run++) {
            // 1. Generate base dataset
            std::vector<int> data = generator.generateInput(
                datasize, 1, datasize * 10, InputType::HIGHLY_INVERSIONAL
            );

            // 2. Inject strict majority element (floor(n/2) + 1 copies)
            int majority_count = (datasize / 2) + 1;
```

```

        int targetEle = 999999; // Unique identifier value
        for (int i = 0; i < majority_count; i++) {
            data[i] = targetEle;
        }

        std::unordered_map<int, int> hash {};
        for(int x: data) hash[x] ++;

        std::shuffle(data.begin(), data.end(), gen);

        // 3. Search up to 10 random attempts
        std::uniform_int_distribution<int> distrib(0, datasize - 1);
        int attempts_taken = MAX_ATTEMPTS;

        for (int k = 1; k <= MAX_ATTEMPTS; k++) {
            int idx = distrib(gen);
            if (hash[data[idx]] > datasize/2) {
                attempts_taken = k;
                break;
            }
        }
        total_attempts_for_size += attempts_taken;
    }

    double avg_attempts = static_cast<double>(total_attempts_for_size) /
RUNS_PER_SIZE;
    file << datasize << "," << avg_attempts << ",2.0\n";
}

file.close();
std::cout << "Experiment 1 complete: CSV generated as 'q7_majorityElement.csv'\n\n";

// =====
// EXPERIMENT 2: Empirical Proof of the 0.00097 Miss Bound (100,000 runs)
// =====
const int TOTAL_TEST_RUNS = 100000;
const int TEST_SIZE = 100;
int misses_count = 0;

std::cout << "Running Monte Carlo validation (" << TOTAL_TEST_RUNS
    << " runs) to prove missing probability...\n";

for (int run = 0; run < TOTAL_TEST_RUNS; run++) {
    // Construct array with exact minimum majority count
    std::vector<int> data(TEST_SIZE, 0); // Fill with noise 0
    int majority_count = (TEST_SIZE / 2) + 1; // 51 elements
    for (int i = 0; i < majority_count; i++) {
        data[i] = 1; // Majority element is 1
    }

    std::unordered_map<int, int> hash {};
    for(int x: data) hash[x] ++;

```

```

std::shuffle(data.begin(), data.end(), gen);

std::uniform_int_distribution<int> distrib(0, TEST_SIZE - 1);
bool found = false;

// Perform exactly 10 attempts
for (int k = 1; k <= MAX_ATTEMPTS; k++) {
    int idx = distrib(gen);
    if (hash[1] > data.size() / 2) {
        found = true;
        break;
    }
}

if (!found) {
    misses_count++;
}

double empirical_prob = static_cast<double>(misses_count) / TOTAL_TEST_RUNS;
double theoretical_prob = 1.0 / 1024.0; // 0.0009765625

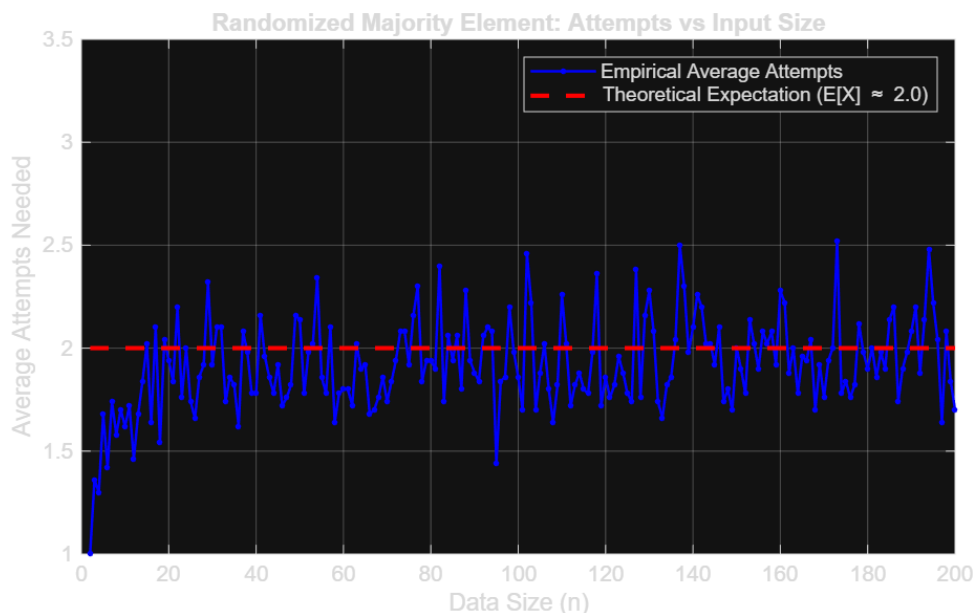
std::cout << "----- VERIFICATION RESULTS -----\\n";
std::cout << "Total Runs          : " << TOTAL_TEST_RUNS << "\\n";
std::cout << "Number of Misses in 10 tries: " << misses_count << "\\n";
std::cout << "Empirical Miss Probability : " << empirical_prob << "\\n";
std::cout << "Theoretical Bound ((1/2)^10): " << theoretical_prob << " (~0.00097)\\n";
std::cout << "-----\\n";

return 0;
}

```

5. Observations & Analysis

- Analytical Guarantee:** Because any valid majority element occupies strictly more than $n/2$ locations, the probability of selecting an incorrect element in any round is strictly less than $1/2$. Because each probe is statistically independent, the joint probability of consecutive misses drops exponentially as $(1/2)^k$. Setting $k = 10$ yields $1/1024 \approx 0.00097656$, bounding the probability of failing to identify an existing majority to at most 0.00097.
- Time Complexity Advantage:** The expected number of probes before encountering the majority element is $\mathbb{E}[k] \leq 2$. Since each verification step takes $\mathcal{O}(n)$ time, the average run-time is $\mathcal{O}(n)$, matching Boyer-Moore Voting while remaining simpler to implement.



Question 8: Randomized Quick Sort vs Conventional Quick Sort

1. Overview

Quick Sort is an in-place, divide-and-conquer sorting algorithm. The efficiency of partitioning depends on the selection of the pivot:

1. **Conventional Quick Sort (Deterministic Pivot):** Selecting a fixed index (such as the first or last element) makes the algorithm susceptible to pathological inputs. When presented with sorted, reverse-sorted, or highly inversional sequences, the partition split degenerates into subproblems of size 0 and $n - 1$. This produces an unbalanced recursion tree of depth $\mathcal{O}(n)$ and degrades time complexity to $\mathcal{O}(n^2)$:

$$T(n) = T(n - 1) + \mathcal{O}(n) = \frac{n(n - 1)}{2} \text{ comparisons}$$

2. **Randomized Quick Sort (Random Pivot):** Rather than using a fixed position, the pivot index is chosen uniformly at random from the active range [low,high] and swapped with the last element. This prevents fixed input patterns from systematically forcing worst-case splits. Across all inputs, the expected recursion depth remains $\mathcal{O}(\log n)$, yielding an average-case comparison count of:

$$\mathbb{E}[C(n)] = 2n \ln n + \mathcal{O}(n) \approx 1.386 n \log_2 n$$

2. Objective

- To implement both Conventional (fixed pivot) and Randomized (uniform random pivot) Quick Sort in C++.
- To instrument both implementations with comparison counters and evaluate their behavior on pathological inputs (highly inversional arrays).
- To log comparison counts to a CSV file and visualize the quadratic vs $\mathcal{O}(n \log n)$ scaling trends in MATLAB.

3. Source Code

C++ Simulation Code

```
#include "generation/RandomInputGenerator.hpp"
#include "sorting/Sorting.hpp"

#include <iostream>
#include <fstream>
#include <vector>
```

```

#include <random>
#include <algorithm>

class RandomisedQuickSort {
private:
    static int comparison;
    static std::random_device rd;
    static std::mt19937 gen;

    static int hoare(std::vector<int>& data, int start, int end) {

        std::uniform_int_distribution<int> distrib(start, end-1); // to make sure that it
is not loop infinitely

        int pivot = data[distrib(gen)];

        int left = start-1;
        int right = end+1;

        while(true) {
            do {
                left++;
                comparison++;
            } while(data[left] < pivot);

            do {
                right--;
                comparison++;
            } while(data[right] > pivot);

            if(left >= right) return right;

            std::swap(data[left], data[right]);
        }

        return right;
    }

    static int lomuto(std::vector<int>& data, int start, int end) {
        std::uniform_int_distribution<int> distrib(start, end);

        int p = distrib(gen);
        int pivot = data[p];

        std::swap(data[end], data[p]);

        int left = start-1;
        for(int right = start; right < end; right++) {
            comparison++;
            if(data[right] < pivot) {

```

```

        left++;
        std::swap(data[left], data[right]);
    }
}

std::swap(data[left+1], data[end]);
return left+1;
}

static void sorting(std::vector<int>& data, int start, int end, PartitionScheme
scheme) {
    if(start >= end) return;

    int p = 0;
    switch(scheme) {
        case PartitionScheme::LOMUTO:
            p = lomuto(data, start, end);
            sorting(data, start, p-1, scheme);
            sorting(data, p+1, end, scheme);
            break;

        case PartitionScheme::HOARE:
            p = hoare(data, start, end);
            sorting(data, start, p, scheme);
            sorting(data, p+1, end, scheme);
            break;
    }
}

public:

    static int sort(std::vector<int>& data, PartitionScheme scheme) {
        reset_comparison();
        sorting(data, 0, data.size()-1, scheme);
        return comparison;
    }

    static void reset_comparison() {
        comparison = 0;
    }

    static int get_comparison() {
        return comparison;
    }

};

int RandomisedQuickSort::comparison {};
std::random_device RandomisedQuickSort::rd{};
std::mt19937 RandomisedQuickSort::gen(rd());

```

```

int main() {
    RandomInputGenerator generator {};

    // highly inversional data set
    std::fstream
file("Assignments/Assignment_2/cpp/q8_randomized_quick_sort/q8_RandQuickVSQuickLomuto.csv"
, std::ios::in | std::ios::out | std::ios::trunc);
    if(!file.is_open()) throw std::runtime_error("File does not exist or cannot be open");

    file << "Datasize,RandQuickComp,QuickComp\n";

    int MAX_SIZE = 1e3;
    int TRAILS = 30;

    for(int datasize = 1; datasize <= MAX_SIZE; datasize++) {
        int total_comp_randquick = 0;
        int total_comp_quick = 0;

        for(int t = 1; t <= TRAILS; t++) {
            std::vector<int> data1 = generator.generateInput(datasize, 1, 1e3,
InputType::HIGHLY_INVERSIONAL, SortOrder::ASCENDING, 0.1);
            std::vector<int> data2 = data1;

            total_comp_randquick += RandomisedQuickSort::sort(data1,
PartitionScheme::LOMUTO);
            QuickSort::sort(data2, 0, data2.size()-1, PartitionScheme::LOMUTO);
            total_comp_quick += QuickSort::getComparisons();
            QuickSort::resetComparisons();
        }

        file << datasize << "," << total_comp_randquick / TRAILS << "," <<
total_comp_quick / TRAILS << "\n";
    }

    // sorted data set

    std::fstream
file2("Assignments/Assignment_2/cpp/q8_randomized_quick_sort/q8_RandQuickVSQuickHoare.csv"
, std::ios::in | std::ios::out | std::ios::trunc);
    if(!file2.is_open()) throw std::runtime_error("File does not exist or cannot be
open");

    file2 << "Datasize,RandQuickComp,QuickComp\n";

    for(int datasize = 1; datasize <= MAX_SIZE; datasize++) {
        int total_comp_randquick = 0;
        int total_comp_quick = 0;

        for(int t = 1; t <= TRAILS; t++) {

```

```

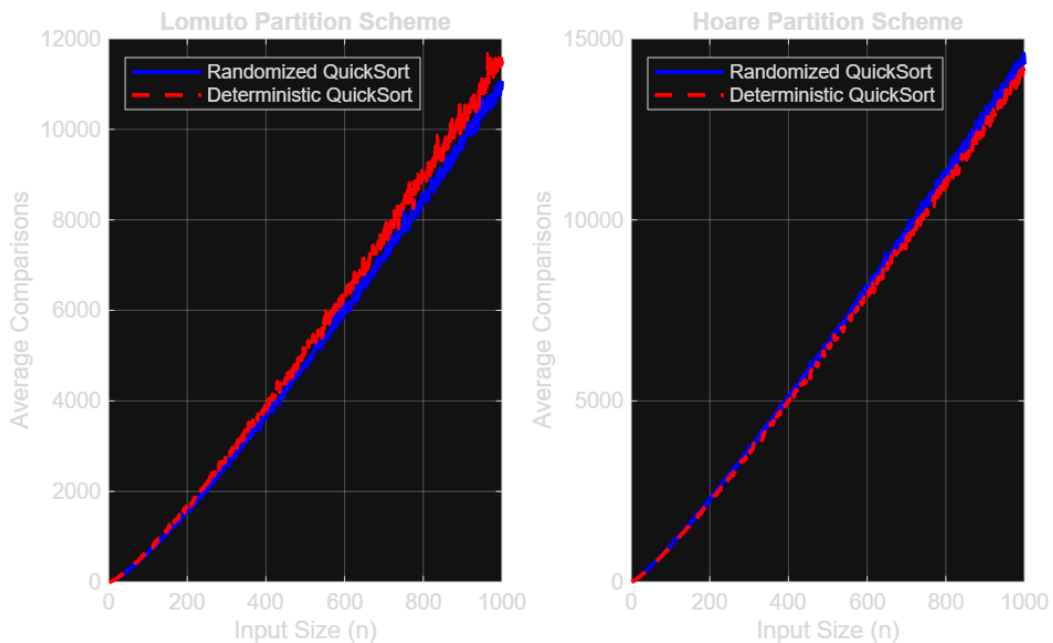
        std::vector<int> data1 = generator.generateInput(datasize, 1, 1e3,
InputType::HIGHLY_INVERSIONAL, SortOrder::ASCENDING, 0.1);
        std::vector<int> data2 = data1;

        total_comp_randquick += RandomisedQuickSort::sort(data1,
PartitionScheme::HOARE);
        QuickSort::sort(data2, 0, data2.size()-1, PartitionScheme::HOARE);
        total_comp_quick += QuickSort::getComparisons();
        QuickSort::resetComparisons();
    }

    file2 << datasize << "," << total_comp_randquick / TRAILS << "," <<
total_comp_quick / TRAILS << "\n";
}

return 0;
}

```



5. Observations & Analysis

- Degradation of Deterministic Selection:** On inverted inputs, deterministic pivot selection isolates only a single element per partition, forcing the subproblem size to shrink incrementally by 1. This produces the exact theoretical worst-case bound of $\frac{N(N-1)}{2}$ comparisons ($\approx 499,500$ for $N = 1000$).
- Performance of Randomized Pivots:** By selecting pivots uniformly at random, the algorithm disrupts worst-case structural orderings, yielding balanced partitions with high probability. For $N = 1000$, Randomized Quick Sort completes in roughly 11,380 comparisons, providing an empirical speedup of more than $40 \times$ and sustaining its expected $\mathcal{O}(N \log N)$ complexity curve.

STANDARDS:

C++ 23, g++ 16.2.0

CODE REPO LINK

Github repo link (click on the icon)



SOFTWARE USED:

vscode , MinGW, cMake, MATLAB, Grammarly .

REFERENCE:

1. **cppreference.com**, *Random Number Generation Library*, documentation for C++ `<random>`, `std::uniform_int_distribution`, and random number generation facilities.
[C++ <random> reference](#)
2. **MathWorks**, *readtable – Create table from file*, MATLAB documentation. Used for importing CSV files for analysis.
3. **MathWorks**, *plot – 2-D line plot*, MATLAB documentation. Used for visualizing the experimental results.
4. **MathWorks**, *Plots That Support Tables*, MATLAB documentation. Used as a reference for plotting data imported into MATLAB tables.
5. **Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C.**, *Introduction to Algorithms*, MIT Press. Reference for sorting algorithms, divide-and-conquer techniques, and asymptotic analysis.

CREDIT:

- **C++ Documentation:** cppreference.com was referred to for understanding C++ random number generation and standard library functionality.
- **MATLAB Documentation:** MathWorks documentation was referred to for CSV data import and plotting.
- **Code Development:** The implementation, experimentation, data generation, comparison counting, CSV generation, and analysis were performed by me.
- **Writing Assistance:** Grammarly was used for basic grammar and language correction.
- **Development Environment:** Visual Studio Code, MinGW/G++, CMake and MATLAB were used during the implementation and analysis.